Error: kernel launch from __device__ or __global__ functions requires separate compilation mode

> cuda

vanellope Nov 2

Hi, I'm beginner. I'm learning CUDA C by working with problem in **Cousera**

Trying to run `make clean build`, this error appeared:

```
coder@c6afbe98144b:~/project$ make clean build
rm -f simple.exe
nvcc -I./ -I/usr/local/cuda/include -lcuda --st
simple.cu(133): error: kernel launch from __dev

simple.cu(233): error: a __global__ function ca

2 errors detected in the compilation of "simple
make: *** [Makefile:8: build] Error 1
```

I looked forward this error but didn't find any document for this error.

Below is system file:
simple.cu:

```
/*
 * Copyright 1993-2015 NVIDIA Corporation.  All
 *
 * Please refer to the NVIDIA end user license
 * with this source code for terms and conditio
 * this software. Any use, reproduction, disclo
 * this software and related documentation outs
 * is strictly prohibited.
 *
 */

/*
 * Vector multiplication: C = A * B.
 *
 * This sample is a very basic sample that impl
 * vector multiplication. It is based on the sa
 * of the programming guide with some additions
 */

#include "simple.h"

/*
 * CUDA Kernel Device code
 *
 * Computes the vector product of A and B into
 * number of elements numElements.
 */
__global__ void vectorMult(const float *A, cons
{
    int i = blockDim.x * blockIdx.x + threadIdx

    if (i < numElements)
```

```
        {
            C[i] = deviceMultiply(A[i], B[i]);
        }
    }

    float deviceMultiply(float a, float b)
    {
        return a * b;
    }

    __host__ std::tuple<float *, float *, float *>
    {
        size_t size = numElements * sizeof(float);

        // Allocate the host input vector A
        float *h_A = (float *)malloc(size);

        // Allocate the host input vector B
        float *h_B = (float *)malloc(size);

        // Allocate the host output vector C
        float *h_C = (float *)malloc(size);

        // Verify that allocations succeeded
        if (h_A == NULL || h_B == NULL || h_C == NU
        {
            fprintf(stderr, "Failed to allocate hos
            exit(EXIT_FAILURE);
        }

        // Initialize the host input vectors
        for (int i = 0; i < numElements; ++i)
        {
            h_A[i] = rand()/(float)RAND_MAX;
            h_B[i] = rand()/(float)RAND_MAX;
        }

        return {h_A, h_B, h_C};
    }

    __device__ std::tuple<float *, float *, float *
    {
        // Allocate the device input vector A
        float *d_A = NULL;
        size_t size = numElements * sizeof(float);
        cudaError_t err = cudaMalloc(&d_A, size);
        if (err != cudaSuccess)
        {
            fprintf(stderr, "Failed to allocate dev
            exit(EXIT_FAILURE);
        }

        // Allocate the device input vector B
        float *d_B = NULL;
        err = cudaMalloc(&d_B, size);
        if (err != cudaSuccess)
        {
            fprintf(stderr, "Failed to allocate dev
```

```
#include <stdio.h>
#include <tuple>

// For the CUDA runtime routines (prefixed with
#include <cuda_runtime.h>

__global__ void vectorMult(const float *A, cons
float deviceMultiply(float a, float b);
__host__ std::tuple<float *, float *, float *>
__device__ std::tuple<float *, float *, float *
__host__ void copyFromHostToDevice(float *h_A,
__global__ void executeKernel(float *d_A, float
__host__ void copyFromDeviceToHost(float *d_C,
__device__ void deallocateMemory(float *h_A, fl
void cleanUpDevice();
__device__ void performTest(float *h_A, float *
```

```
    // Copy the host input vectors A and B in n
    printf("Copy input data from the host memor
    cudaError_t err = cudaMemcpy(d_A, h_A, size
IDIR=./
COMPILER=nvcc
COMPILER_FLAGS=-I$(IDIR) -I/usr/local/cuda/incl

.PHONY: clean build run

build: simple.cu simple.h
    $(COMPILER) $(COMPILER_FLAGS) simple.cu -o

clean:
    rm -f simple.exe

run:
    ./simple.exe

all: clean build run
```

```
    int blocksPerGrid =(numElements + threadsPe
    printf("CUDA kernel launch with %d blocks o
```

> ✅ Solved by **Robert_Crovella** in **post #2**
>
> You're evidently confused about the decorators __global__, __device__ and when to use them. __global__ is used to mark a kernel definition only. It indicates code that will run on the device. __device__ (by itself, when used to mark code) is used to mark code that will run on the device, but is n...

Robert_Crovella Moderator — **Nov 2**

You're evidently confused about the decorators `__global__`, `__device__` and when to use them.

`__global__` is used to mark a kernel definition only. It indicates code that will run on the device.

`__device__` (by itself, when used to mark code) is used to mark code that will run on the device, but is not a kernel by itself.

`vectorMult` should be marked with `__global__`, what you have there is correct.

`deviceMultiply` is called from device code (from the `vectorMult` kernel.) it must be marked with `__device__`.

`allocateDeviceMemory` is called from host code and is expected to run on the host. It should not be marked with `__device__`.

`executeKernel` is host code. It should not be marked with `__global__`.

`deallocateMemory` is host code. It should not be marked with `__device__`.

`performTest` is host code. It should not be marked with `__device__`.

Nvcc does not understand __global__

dc_vector **Nov 3**

This was completely brilliant! Exactly what I needed to get my code to compile and run neatly. Thank you very much.

vanellope **Nov 4**

Thanks @Robert_Crovella for straight forward answer.
AFAIK, it's simply just look at the code in main() and see which function called from `main()` should be marked with decorator `__host__` whereas the function is called to perform calculation marked with `__global__` and its child components is marked with `__device__`, Right?

Robert_Crovella Moderator **Nov 4**

That's right. For the purpose of this discussion, at this level of understanding, the kernel launch syntax:

```
kernel_name<<<...>>>(...)
```

is the dividing line. The function defined with the name `kernel_name` needs to be decorated with the `__global__` keyword. Anything called from `kernel_name` needs to be decorated with the `__device__` keyword. Repeat for every kernel launch. Everything else can remain undecorated (which is equivalent to `__host__` decoration, when using `nvcc`). This doesn't take into account CUDA Dynamic Parallelism, and possibly other advanced topics (CUDA Driver API, other launch methods such as CG, decorating a function with both `__host__` and `__device__` etc.), but these topics are not relevant to the discussion at this level (beginner level - according to your own statement.)

vanellope **Nov 4**

I'm reading 3 first chapter of `Programming Massively Parallel Processors: A Hands-on Approach 4th edition`, it's quite easy to understand for beginner like me. They go deep to distinguish 3 kind of

different decorators in CUDA C. Any tips for beginner to read and practice code?

---

rentsrule **Nov 6**

Hi, I also take the course.
To do “make”, i needed to modify main function the simple.cu file;
int threadsPerBlock = 256;
int blocksPerGrid =(numElements + threadsPerBlock - 1) / threadsPerBlock;
copyFromHostToDevice<<<blocksPerGrid, threadsPerBlock>>>(h_A, h_B, d_A, d_B, numElements);

I think this is ok and seems current standard description.
But ,
cleanUpDevice();
in the main function makes an error;
simple.cu(244): error: a **global** function call must be configured

Actually, the cleanUpDevice is global function and device must do it, and **should not** be called on host.
So, I do not know how fix this error.

Robert_Crovella Moderator **Nov 6**

> rentsrule:
>
> simple.cu(244): error: a **global** function call must be configured
>
> Actually, the cleanUpDevice is global function and device must do it, and **should not** be called on host.

No, that is incorrect. You probably have a `__global__` keyword decorating your `cleanUpDevice` function. The OP for this thread does not have that. If you have that, its incorrect. You should remove that decorator from `cleanUpDevice`.

The device must ***not*** do that function.

---

rentsrule **Nov 7**

Thank you, i understand about the decoration.
Below decoration makes pass test
global void vectorMult
device float deviceMultiply
host std::tuple<float *, float *, float *> allocateHostMemory
host std::tuple<float *, float *, float *> allocateDeviceMemory
host void copyFromHostToDevice
host void executeKernel
host void copyFromDeviceToHost
host void deallocateMemory
host void cleanUpDevice
host void performTest

Closed on Nov 21, 2023

This topic was automatically closed 14 days after the last reply. New replies are no longer allowed.

New & Unread Topics

Topic	Replies	Views	Activity
☑ Effect of launch bounds on register usage and spillage	2	7	4h
☑ Error: Number of CUDA devices (2) is less than MPI processes (4)	1	12	Aug 19
You have exhausted the lab time provided with your course enrollment dli	0	11	Aug 18
How to copy integer from Device to Host? cuda kernel	1	659	Sep 3
Using DirectStorage with Cuda on Windows 10 onwards cuda	0	234	Sep 14
An application compiled using `nvcc` does not show its dependency on the `libcuda.so` library with `ldd` linux command	4	454	Sep 15
Why 2 GPUs is slower than 1 GPU cuda kernel	6	417	Dec 4
Programming Tensor core in RTX4070	1	383	Jan 19
Persistant L2 API restrict in stream, what if in other stream?	3	359	Feb 2
The order of CTA execution	5	348	Apr 11

There are 122 new topics remaining, or browse other topics in